Part II

Q) What is a Virtual Machine (VM)? How is running a program on a VM different from running locally?

A) A VM is a computer running in the cloud on shared physical hardware. Running a program on a VM is similar to local, but we can access it remotely using the SSH protocol, it has cloud networking, firewalls, different hardware specs, and network latency affects requests.

Q) You allowed SSH only from your IP. What if your IP changes?

A) We must update the EC2 Security Group inbound rule for port 22 to our new public IP or use a wider range like 0.0.0.0/0 temporarily, but that’s less secure.

Q) Why does EC2 get a different IP after stop/terminate/restart? How does Elastic IP help?

A) If we stop/start an instance, AWS can assign a new public IPv4. An Elastic IP is a static public IP you allocate and attach to your instance so the public IP stays the same across stops/starts.

Q) What level of service are you “paying” for with your instance type? Why care about specs?

A) I used a t2.micro instance, which is a low-cost VM. It has limited memory so under sustained load it can slow down. Specs matter because they directly affect performance, stability under load, and cost.

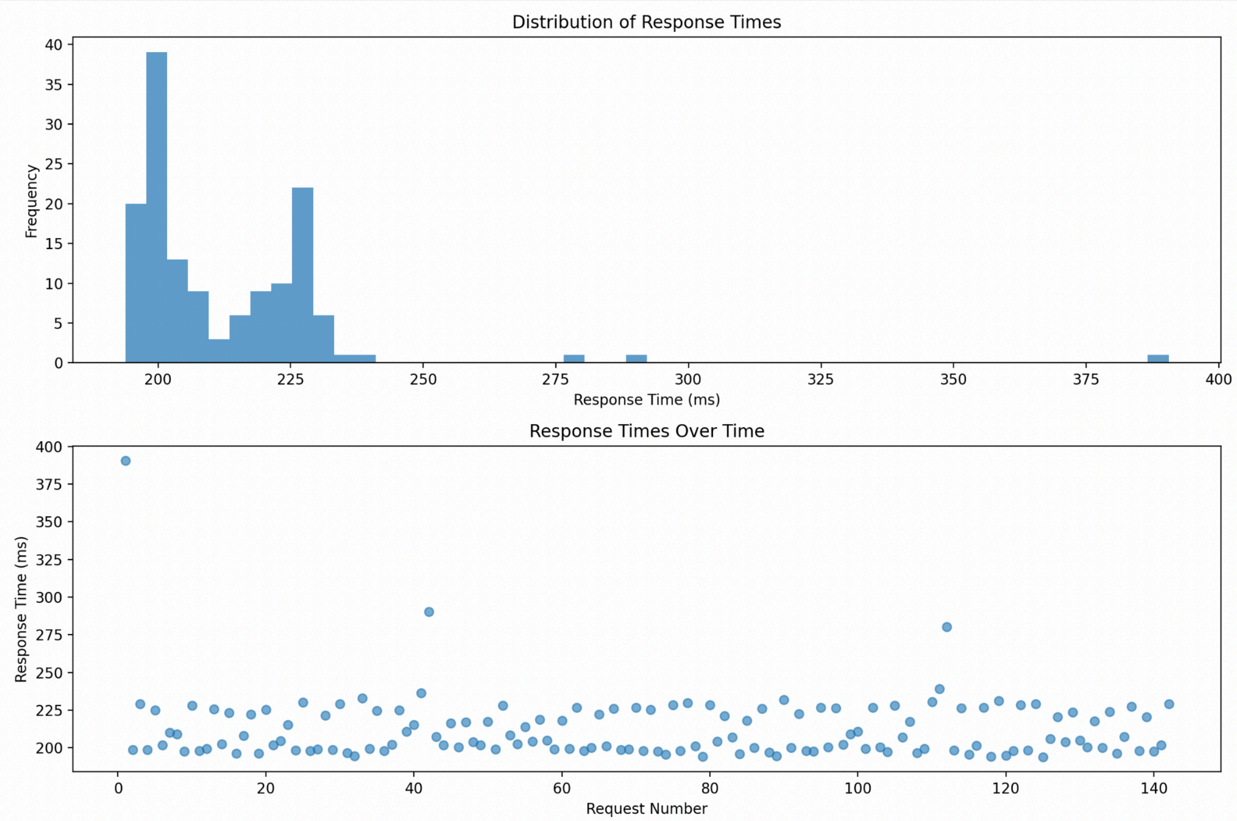
Q) What was different between getting your backend going on GCP vs AWS?

A) Both have SSH, firewall rules, and exposing the service port. Differences: AWS uses Security Groups, VPC, Subnets, key pairs for SSH are emphasized, and public IP behavior is more explicit. GCP was more straightforward and easy to set up.

Q) What testing did you do, and how did you do it?

A) I deployed the Go service on EC2 and wrote a Python load-test script that sent repeated GET requests for 30 seconds to public IP of the EC2 instance, recorded response times, then plotted a histogram and scatter plot and computed the avg/median/p95/p99 to understand latency under load.

Part III



1. Distribution Shape: Most responses cluster around ~190–235 ms. There's a small right tail with a few slow outliers (~280–390 ms). "Slow" (>250 ms) looks like only a few requests.

2. Consistency: Mostly stable around ~200–225 ms, with occasional spikes.

3. Percentiles: Median is around ~200 ms, but p95 is higher because of the tail. The median p95 gap shows some outliers.

4. Infrastructure Impact: On a t2.micro, small CPU credit effects and background work can cause random latency spikes.

5. 100 concurrent users: Latency would increase, and p95 would get worse due to CPU limits, might see timeouts if overloaded.

6. Network vs processing: Baseline is network and server time, spikes could be network jitter or server pauses.